

COS 214 REPORT

Contents

Team Information	2
Research Brief	2
References	4
Functional Requirements.....	4
NON-FUNCTIONAL REQUIREMENTS	6
DESIGN PATTERN APPLICATION	7
ALL DESIGN PATTERNS	7
GREENHOUSE	7
STAFF	12
CUSTOMER AND SALES	18
GitHub Branch Strategy.....	22
GitHub Actions.....	23
State Diagram.....	23
Activity Diagram.....	24
OBJECT DIAGRAM.....	24
COMMUNICATION DIAGRAM.....	25
SEQUENCE DIAGRAM.....	25
SYSTEM CLASS DIAGRAM	26
High LEVEL Activity DIAGRAM.....	27

TEAM INFORMATION

Multimedia Adapters (MM Adapters) https://github.com/u23527685/MMAdapters.git		
Name	Student Number	Github
Jerusha Thaver	u23686376	https://github.com/Jerusha-Thaver https://github.com/JerushaThaver
Cobus Botha	u23556502	https://github.com/cobusbotha
Isha Kalyan	u24575977	https://github.com/RebelsReverie27 https://github.com/isha27kalyan-cloud
Jay Lopes	u23576066	https://github.com/u23576066
Okaile Gaesale	u23527685	https://github.com/u23527685
Mthokozisi Duba	u24690059	https://github.com/Mthokozisi-Duba
Rachel Kambala	u23559129	https://github.com/rachel-m-kambala

RESEARCH BRIEF

Our research focused on understanding the key components of managing a nursery, including plant types and their care. We discovered that fertilizers can be broadly classified into organic and inorganic categories, each with its unique benefits and drawbacks according to Vasyi Cherlinka (Cherlinka 2023). For fertilizers, we measured the amounts in parts per million (ppm) as recommended by (Wong 2025). We understand that different plants need different amounts of sunlight and water and therefore added different plant care strategies to cater for this to ensure plants get perfect care.

We also considered different plant categories, including Tropical, Sunny, Shaded, and Temperate, to ensure the right category is picked according to the plant's specific care requirements.

Our research on plant care and climate zones, as outlined in (Vu 2025), informed our understanding of the specific care requirements for plants in different categories. We used this knowledge to determine the optimal care routines for plants in each category, including the right amount of sunlight, water, and fertilizer.

Staff training in plant care and customer service is vital for the success of a nursery. Employees not only maintain plant health but also serve as key touchpoints for customer engagement (JAENICKE 2004) (Anthony Mbora 2008). The interaction between staff and customers influences purchasing decisions and brand reputation. Nurseries that foster knowledgeable, approachable staff often achieve higher sales and stronger customer relationships. (Hayk Khachatryan 2021) (Susan S. Barton 2002)

In addition to academic sources, we also drew inspiration from online resources. A YouTube video discussing the Builder design pattern provided valuable insight into using this pattern to create objects with multiple constructors, simplifying the construction process. This influenced our decision to implement the Builder pattern in our design. (The Builder Pattern Explained and Implemented in Java n.d.)

Our research influenced the following design decisions

- Climate-Based Plant Categorization: We added categories for different climates, including Tropical, Sunny, Shaded, and Temperate, to ensure the right strategy for the plants are selected according to their specific requirements, as outlined in [Vu \(2025\)](#).
- Plant Care Strategy: We incorporated organic and inorganic fertilizer options, different water levels, and varying sunlight percentages to accommodate different plant needs and preferences, as informed by [\(Cherlinka 2023\)](#). This comprehensive approach allows us to provide tailored care to each plant, taking into account its unique requirements for nutrients, water, and sunlight.
- State Pattern: We implemented the State pattern to simulate the growth and changes in plants as they receive care, mirroring real-life plant development and responses to care.
- Observer Pattern: We implemented the observer design pattern to notify staff when a plant's health levels drop, ensuring timely care and intervention to restore the plant's health. This allows for continuous monitoring and care of plants as they grow and change.
- Plant Inventory Management: Singleton ensures a single, centralized instance of the plant inventory, allowing consistent tracking of all plants across the system.
- System Structure and Access: The Facade simplifies main testing interactions with the system, while the Proxy controls secure access to critical functions, preventing customers from directly accessing the greenhouse.
- Object Creation: The Factory Method, Strategy, and Decorator patterns enabled flexible creation of plant objects and customisation of care options without impacting existing plants.
- User Queries: Builder Pattern constructs structured queries of customers to staff, specifying plant type, and care request, while the Chain of Responsibility routes and handles these queries efficiently

Assumptions

- We assume users have a basic understanding of plant care and nursery operations.
- We assume all plants are categorised under Tropical, Sunny, Shaded, or Temperate groups.
- We assume environmental inputs (water, sunlight, fertilizer) are measurable and consistent.
- We assume plant health can be represented numerically and responds predictably to care actions.
- We assume that system users will maintain regular updates on plant conditions.
- Plant health is assumed to be quantifiable and influenced predictably by variations in care inputs (if they receive care such as being watered for example)
- It is assumed that nursery staff will need to update plant data and monitor plant conditions through the system.
- Users are assumed to interact with nursery staff effectively for support or additional plant care advice.

Definitions

- Plant Care Strategy: The combination of water, sunlight, and fertilizer given to a plant according to its needs.
- Climate Categories: Classifications of plants by environmental needs, Tropical, Sunny, Shaded, and Temperate.
- Factory Method Pattern: Used to create specific plant objects (e.g., rose for a flower or oak for tree) without specifying exact classes in the main code.

- State Pattern: Models plant growth or health changes as care inputs vary.
- Observer Pattern: Alerts staff when a plant's health drops, enabling prompt attention.
- Plant Catalog: The digital listing of plants available in the nursery, including details such as available stock in store, name and price.
- Purchase Process: The sequence of steps a customer follows to buy a plant, including selection, payment, and receipt.
- Interaction with Staff: Communication between the user and nursery personnel for guidance, care instructions, or assistance.

References

- Anthony Mbora, Jens-Peter B. Lillesø, and Ramni Jamnadass. 2008. *Good Nursery Practices: A Simple Guide*. World Agroforestry Centre (ICRAF).
- Cherlinka, Vasyl. 2023. *Types of Fertilizers and How to Pick the Right One*. Accessed October 29, 2025. <https://eos.com/blog/types-of-fertilizers/>.
- Hayk Khachatryan, Alicia Rihn, Xuan Wei. 2021. *Plant selection behavior and promotion use by garden center customers*. <https://edis.ifas.ufl.edu/publication/FE1098?>
- JAENICKE, HANNA H. 2004. *Good tree nursery practices: Practical guidelines for research nurseries*. International Centre for Research in Agroforestry (ICRAF).
- Susan S. Barton, J.R. Brooker, C.R. Hall, S.C. Turner. 2002. *Review of customer preference research in the nursery and landscape industry*. Journal of Environmental Horticulture.
- n.d. *The Builder Pattern Explained and Implemented in Java*. Prod. Geekific. YouTube. Accessed October 31, 2025. https://www.youtube.com/watch?v=MaY_MDdWkQw.
- Vu, Q. 2025. *How to Choose Plants for Different Climates*. Accessed October 29, 2025. https://www.xroci.com/blogs/xroci-guide-and-tips/how-to-choose-plants-for-different-climates?srsId=AfmBOoh5DaDvtQte_fIUfYn36XbYckWFP1Q3ZvkRxa-fmGwb4YAIJTA.
- Wong, Dennis. 2025. *How to Read Fertilizer Labels, and What Is ppm?* Accessed October 29, 2025. <https://www.2hraquarist.com/blogs/fertilize-planted-tank/how-to-read-fertiliser-labels?srsId=AfmBOopYW0RJ9gG5QdpS53CtYf6GrQHfESHltoa2ID4WU-5pjIBXocFH>.

FUNCTIONAL REQUIREMENTS

Greenhouse		
FR No.	Functional Requirement	Design Pattern
1	The system must allow plants to dynamically apply different care strategies (watering, sunlight, fertilizer) based on plant types category.	Strategy Pattern (Plant Care)
2	The system must allow a plant to transition between different life cycle stages – Seed, Seedling, Mature, Distressed, and Withered – based on environmental events such as day count, sunlight level, water level and fertilizer levels.	State Pattern (Plant Life Cycle)
3	The system must notify all observers (e.g. staff Observer and salesfloor Observer) whenever plant inventory is updated	Observer Pattern (Inventory Tracking)
4	The system shall allow the addition of new stock to the greenhouse inventory through a command object (Stock manager), enabling dynamic execution and potential	Command Pattern (Stock Management)

	queuing of stock addition requests without modifying existing code. This ensures seamless integration with plant creation and inventory updates as the nursery grows.	
5	This system must create different categories of plants (such as trees, flowers) using specific factory classes that produce plant object (e.g Rose) with their corresponding care requirements based on category attribute.	Factory Method Pattern (Plant Types)
6	The system must ensure that only one instance of the class is created.	Singleton (Inventory)

Staff		
FR No.	Functional Requirement	Design Pattern
1	The system must allow staff to perform plant care routines where general steps are predefined but should allow for specific care routines for specific plant types.	Template Method Pattern (Plant Care Routine)
2	The system should notify staff automatically whenever a plant's growth stage or health status change, so appropriate action can be taken. It notifies when the plant life cycle changes and the plant is in the mature state and available to be sold.	Observer Pattern (Staff–Inventory Communication works with Plant Life Cycle)
3	The system must allow staff to assist customers with queries. Each staff member has different roles and what it is able to handle.	Chain of Responsibility Pattern (Interaction with staff)
4	The system will create different types of staff members based on their assigned responsibilities	Factory Method Pattern (Staff Roles)
5	The system shall ensure that all created staff objects conform to a common interface to maintain consistency in staff management operations.	Abstract Factory (Staff Types)

Customer & Sales		
FR No.	Functional Requirement	Design Pattern
1	The system shall allow customers to personalize their purchase with decorations such as pots or wrapping.	Decorator Pattern (Personalisation)
2	The system shall record transaction history to allow retrieval of past purchases.	Memento Pattern (Transaction History)

3	The system shall allow customers to choose different payment methods (e.g. EFT, E-Wallet, Cash).	Strategy Pattern (Payment Method)
5	The system shall maintain an Inventory as the subject that keeps track of all plants in stock and allow observers staff to subscribe to inventory updates	Observer Design Pattern (Updating Inventory)
6	Customers interact with staff to ask for advice or help with purchases	Chain of Responsibility (Interaction with Customers & Staff)
7	The system shall construct customer queries for plant information, enabling flexible object creation without relying on multiple constructors.	Builder (Building Customer Queries)
8	The proxy shall restrict customers from performing unauthorized actions such as adding or removing stock items.	Proxy (Customer Restrictions)

NON-FUNCTIONAL REQUIREMENTS

NFR No.	Non-functional Requirement	Quality Attribute
1	Customer and staff information should be kept safe and private. Customer can not be able to access actions staff can only do (e.g. add to stock and remove to stock).	Safety
2	New plant types, payments options, additional plant decorations or staff roles can be easy to add later.	Maintainability / Extensibility
3	The system is easy to use (use of menus) and understand for new users (tutorial on the system in the beginning).	Usability
4	The system shall respond to user inputs within an acceptable time frame and utilize minimal system resources, suitable for low-spec environments.	Efficiency

DESIGN PATTERN APPLICATION

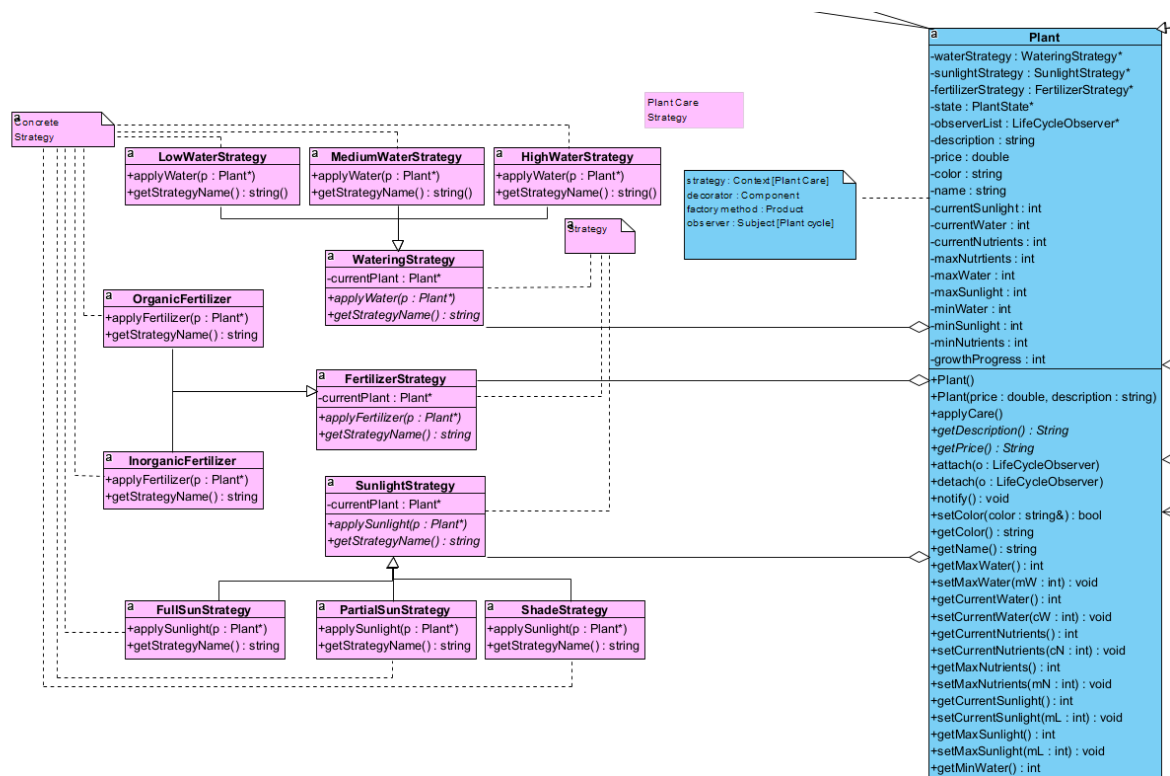
ALL DESIGN PATTERNS

1. Memento
2. Strategy
3. Decorator
4. State
5. Abstract Factory
6. Factory Method
7. Template Method
8. Chain of Responsibility
9. Observer
10. Command,
11. Singleton
12. Builder
13. Proxy

Note: Some patterns are used multiple times

GREENHOUSE

1 Strategy Design Pattern for Plant Care



The Strategy design pattern is used in the Plant Care system to manage different care routines for various plants.

This pattern is chosen for several reasons:

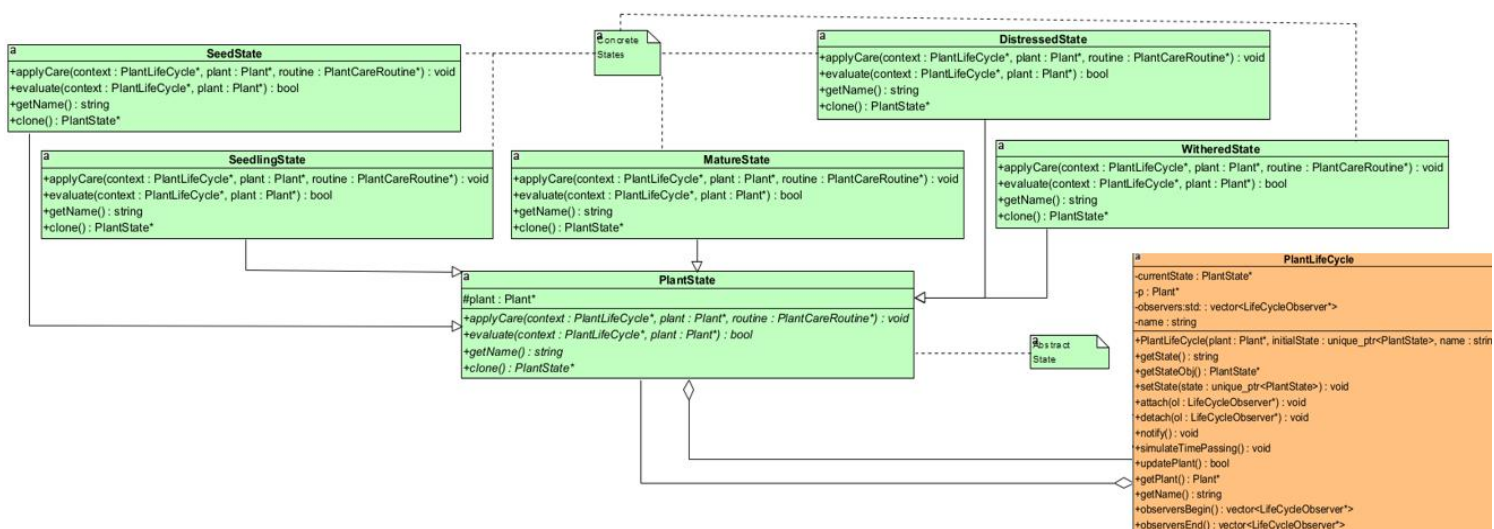
- **Variability in Plant Care:** Different plants have distinct care requirements, including varying amounts of water, sunlight exposure, and fertilizer. The Strategy pattern allows us to define a family of algorithms (care routines) that can be used interchangeably to accommodate these varying needs.
- **Promotes flexibility in managing plant growth:** By using the Strategy pattern, we can add new care strategies or modify existing ones without affecting the underlying plant management logic, making the system more flexible and easier to extend.

- Reduces complexity in plant management: The Strategy pattern helps to reduce complexity by encapsulating specific growth characteristics, making the code more organized and easier to maintain.

Justification

The Strategy pattern is ideal for plant care management due to its ability to encapsulate varying care requirements into interchangeable algorithms. By decoupling plant care logic from specific routines, we achieve a flexible and scalable system that can adapt to diverse plant needs, making it easier to add new care strategies or modify existing ones.

2 State Design Pattern for Plant Lifecycle



The State design pattern is used to manage the life cycle of plants in the nursery system.

This pattern is chosen for several reasons:

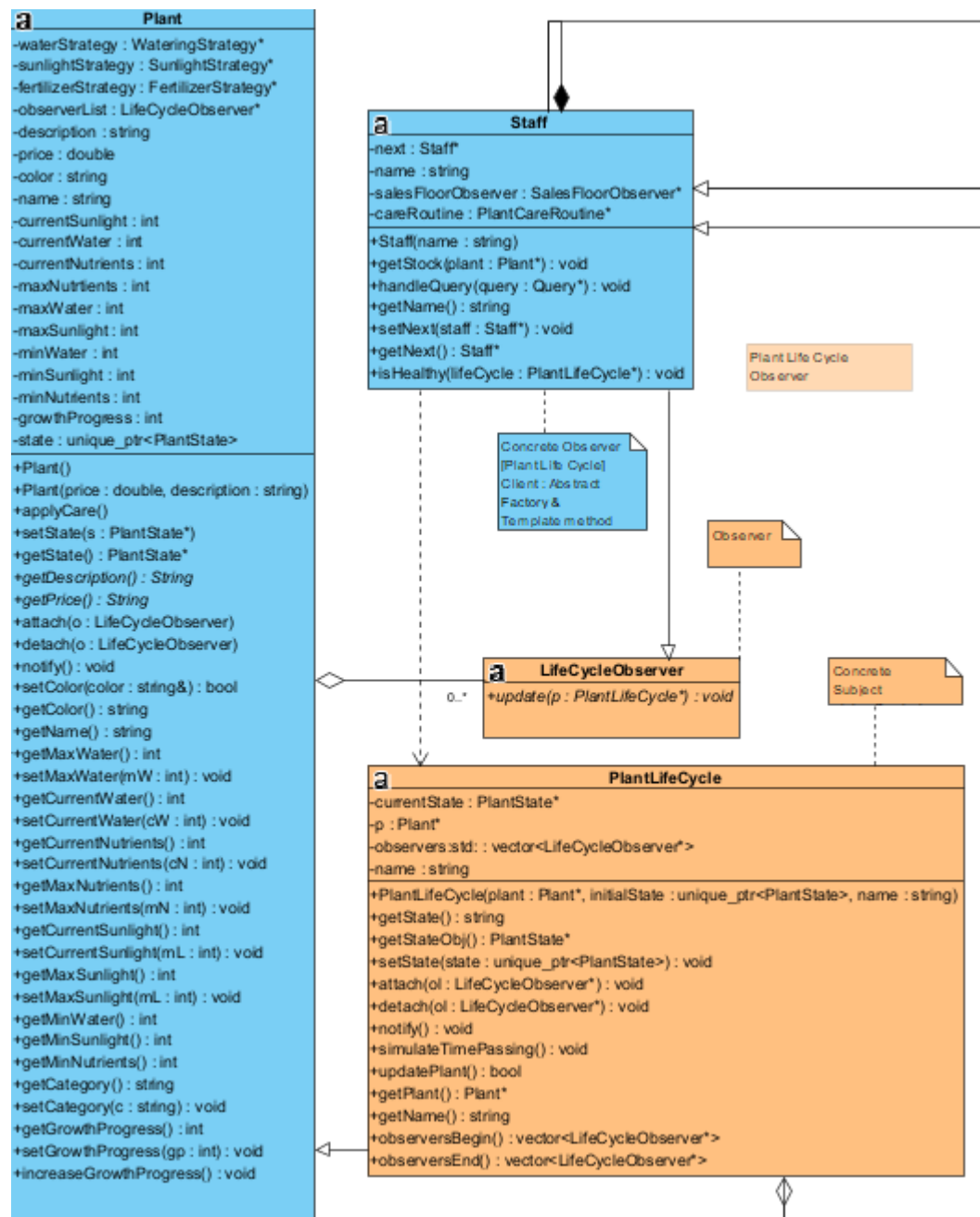
- **Dynamic State Changes:** The State pattern allows plants to change their behaviour based on their current state (e.g., Seedling, Mature, Distressed, Withered). This enables the system to respond dynamically to changes in the plant's condition.
- **Encapsulation of State-Specific Behaviour:** Each state (Seedling, Mature, Distressed, Withered) encapsulates its own behaviour, making it easy to add new states or modify existing ones without affecting other parts of the system.
- **Improved Code Organization:** The State pattern promotes code organization by separating state-specific logic into distinct classes, making the code more maintainable and easier to understand.
- **Easier Extension:** The State pattern makes it easy to add new states or transitions between states without modifying existing code, allowing for greater flexibility and extensibility.
- **Benefits in the Context of the Nursery System:**
- **Accurate Representation of Plant Life Cycle:** The State pattern accurately represents the different stages of a plant's life cycle, enabling the system to respond accordingly.

Justification

The State pattern is used to manage the plant lifecycle, allowing for dynamic state changes and encapsulation of state-specific behavior. This approach enables accurate

representation of a plant's life stages, from Seedling to Withered, and promotes code organization, maintainability, and flexibility. By separating state-specific logic into distinct classes, the system can easily adapt to changing plant conditions and be extended with new states or transitions.

3 Observer design pattern to monitor Plant Lifecycle



The Observer pattern is ideal for this scenario because it allows for loose coupling between the subject (Plant) and its observers (Staff). The Plant object doesn't need to know about the specific staff members or their notification preferences. Instead, it simply notifies its observers when its state changes.

This pattern is chosen for several reasons:

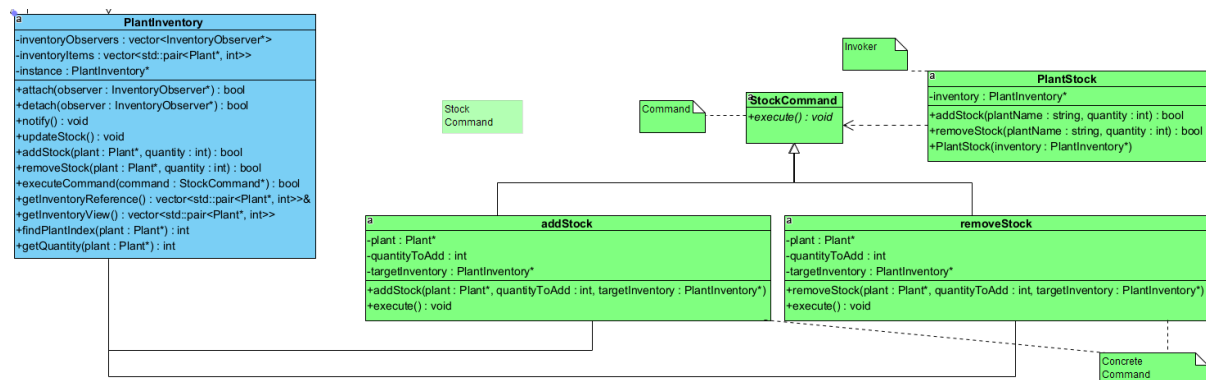
- **Decoupling:** The Plant object is decoupled from the Staff objects, making it easier to add or remove observers without affecting the Plant class.

- Extensibility: New types of observers (e.g., automated systems or other staff members) can be added without modifying the Plant class.
- Reusability: The Observer pattern makes the Plant class more reusable, as it can be used in different contexts with different observers.
- Efficient Notification: Observers are notified only when the Plant's state changes, reducing unnecessary notifications.

Justification:

In the nursery system, plants can change state (e.g., from healthy to distressed or withered) due to various factors. Staff need to be notified promptly when a plant's state changes so they can take action. The Observer pattern provides a clean and efficient way to implement this notification mechanism.

4 Command design pattern for managing Stock



The Command pattern is used to manage stock operations in a flexible and extensible way.

This pattern is chosen for several reasons:

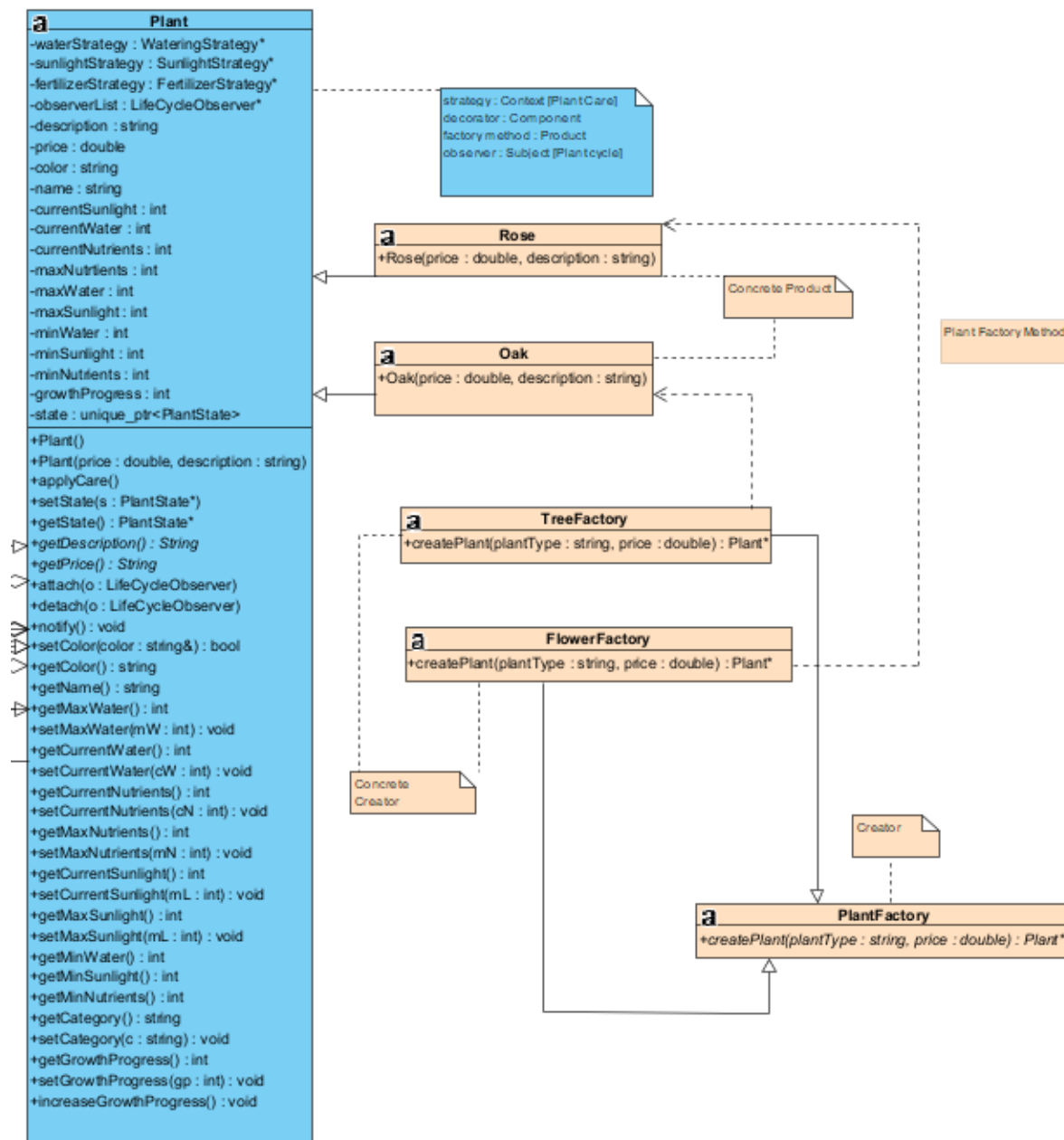
- Decoupling: The Command pattern decouples the client of a stock operation (e.g., PlantStock) from the actual action of the operation (e.g., PlantInventory). This makes it easier to change or replace either component without affecting the other.
- Encapsulation: Adding or removing stock involves multiple steps, such as updating inventory levels and notifying observers. The Command pattern encapsulates these complex operations within a single object, making the code easier to understand and maintain. This makes it easier to modify or extend the behaviour of the system.
- Extensibility: The Command pattern makes it easy to add new stock operations (e.g., transferring stock between locations) by creating new concrete command classes.
- Reusability: The Command pattern promotes reusability by allowing the same command class to be used in different contexts.
- In the context of the requirement "New stock should be easily added as the nursery grows," the Command pattern provides a flexible and extensible way to manage stock operations, making it easier to add new stock or modify existing stock management logic as the nursery grows.

Justification

Stock activities are managed using the Command pattern, which offers a versatile and extendable approach to handling complex tasks like adding or removing stock. The Command pattern is especially well-suited for controlling stock activities because, in contrast to other patterns, it may split the requester from the receiver and encapsulate complicated processes within a single object. This method is perfect for a developing nursery that needs

a scalable and maintainable inventory management system since it makes it simple to add new stock actions, modify current ones, and reuse command logic across various contexts.

5 Factory Method design pattern for plant creation



The Factory Method pattern is used to create plants of different types (e.g., flowers, trees). The **PlantFactory** class is an abstract base class that defines the interface for creating plants, and the **FlowerFactory** and **TreeFactory** classes are concrete subclasses that implement the **createPlant** method to create specific types of plants.

This pattern is chosen for several reasons:

- **Decoupling:** The client code (e.g., the code that uses the **PlantFactory** class) is decoupled from the specific classes of plants being created. This makes it easier to add new types of plants without modifying the client code.
- **Extensibility:** The Factory Method pattern makes it easy to add new types of plants by creating new concrete factory classes.

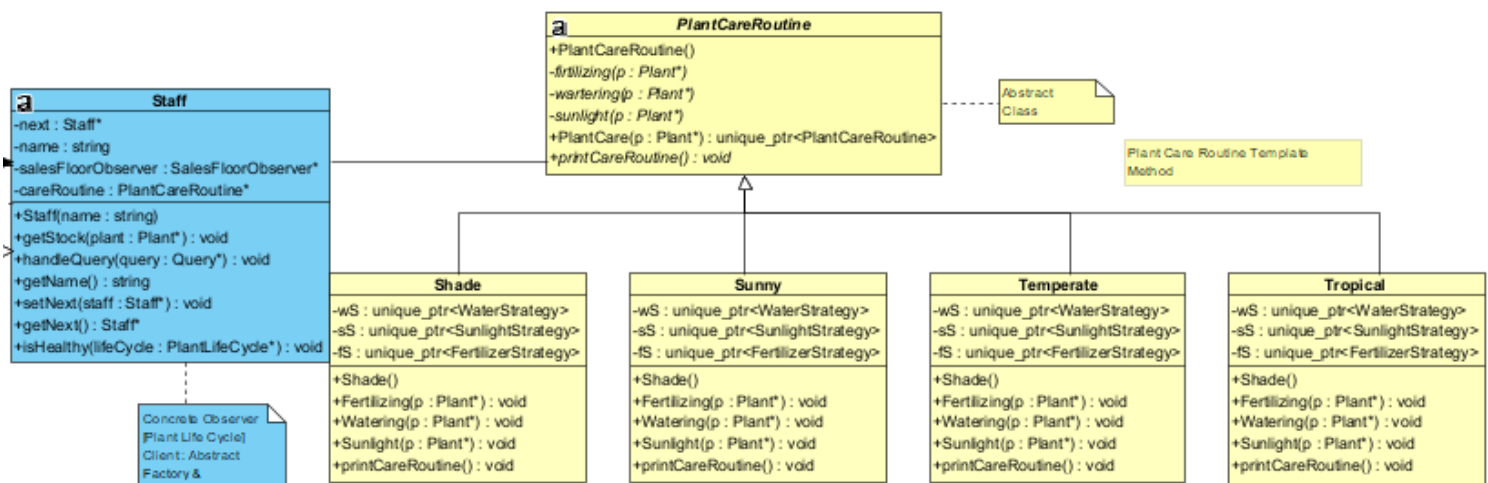
- Polymorphism: The PlantFactory class can return objects of different classes (e.g., Rose, Oak), which can be treated polymorphically as Plant objects.

Justification

The Factory Method pattern is particularly well-suited for the nursery system, where various types of plants need to be created, such as flowers and trees. By using this pattern, we can encapsulate the creation logic for each plant type, allowing for easy extension and modification of plant types without affecting the underlying system. This flexibility is crucial in a nursery setting, where new plant species and varieties are constantly being introduced, and the system needs to adapt quickly to changing inventory needs.

STAFF

6 Template Method design pattern for Plant Care Routine



The Template Method pattern is used to define a common care process for plants, allowing specific steps to be customized by subclasses. This approach enables the creation of plant-specific care routines.

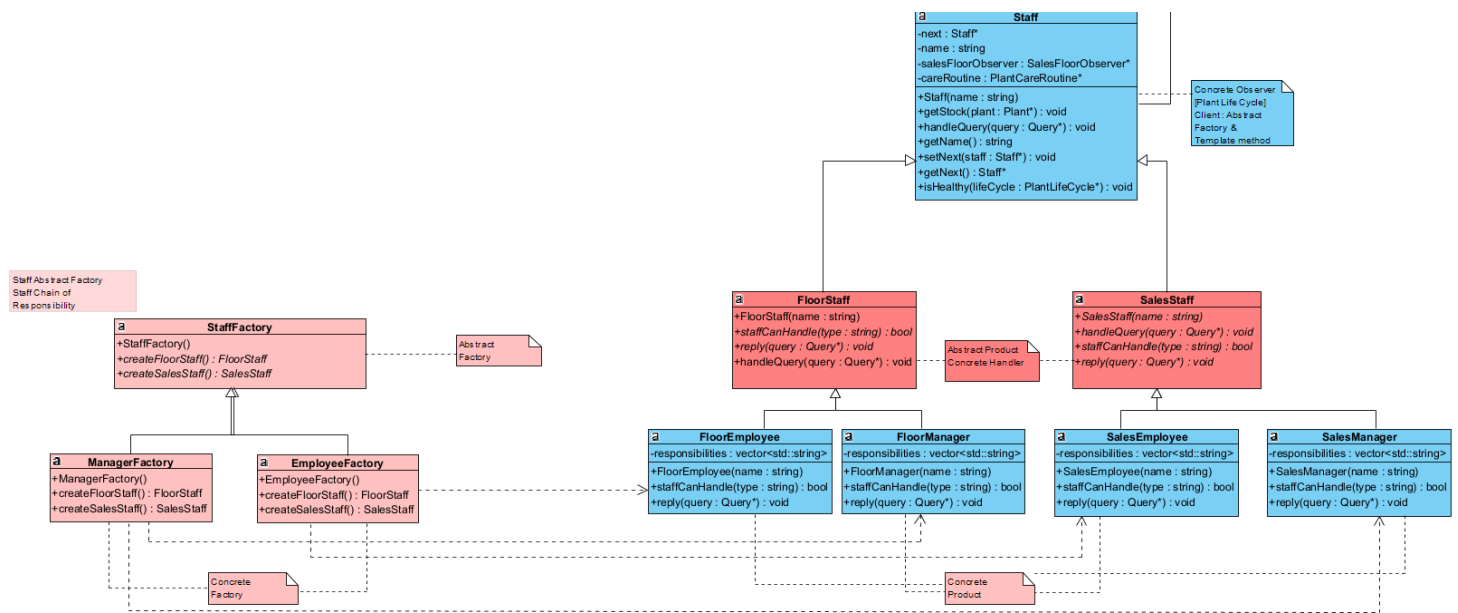
This pattern is chosen for several reasons:

- Code Reuse and Standardisation:** The overall algorithm for plant care is defined once in the base class, ensuring all care routines follow the same high-level steps. This avoids code duplication.
- Flexibility:** The structure is fixed but the specific implementation for each step is delegated to subclasses, allowing vast differences in care between plants.
- Maintainability:** Changing the overall care process only requires a modification in the base class.

Justification

The Template Method pattern is ideal for plant care management because it defines a common care process with customizable steps, this pattern enables efficient code reuse, ensures consistency across different plant types, and allows for easy maintenance and modification of the care process, making it well-suited for managing diverse plant care needs.

7 Abstract Factory design pattern for Staff Roles



The Abstract Factory pattern is used to create families of related staff objects, allowing for the creation of different types of staff members with varying attributes and permissions

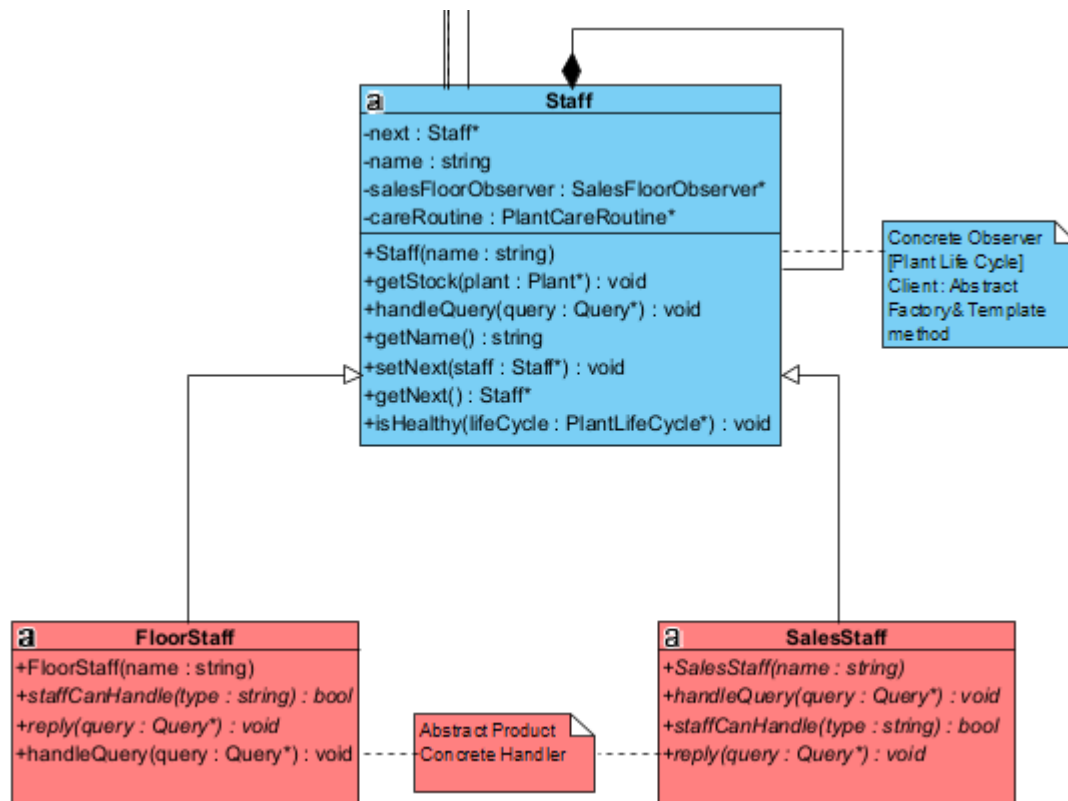
This pattern is chosen for several reasons:

- **Consistency Across Roles:** This pattern guarantees the compatibility of the workers produced by a factory.
- **Decoupling Client Code:** The system only depends on the abstract factory interface, not the concrete staff classes, when it requires a team of employees. This facilitates the transition between “senior” and “junior” teams.
- **Scalability:** The system may be made to scale gracefully by adding new roles to the abstract factory and all of its concrete implementations as the nursery expands.
- **Separation of employee departments:** The pattern allows to different kinds of employees between different departments (e.g Staff and Floor). This reduces the amount of classes and code.

Justification

The Abstract Factory pattern is ideal for staff creation in the nursery system because it ensures consistency across related staff roles, decouples client code from specific staff implementations, and provides a scalable foundation for adding new roles or teams as the organization grows, making it a robust solution for managing diverse staffing needs.

8 Chain of Responsibility (Customer Interaction with Staff)



The Chain of Responsibility pattern is used to handle customer queries by passing them through a chain of staff members, allowing each member to decide whether to handle the query or pass it on to the next member with the appropriate knowledge. Each staff member can have different responsibilities.

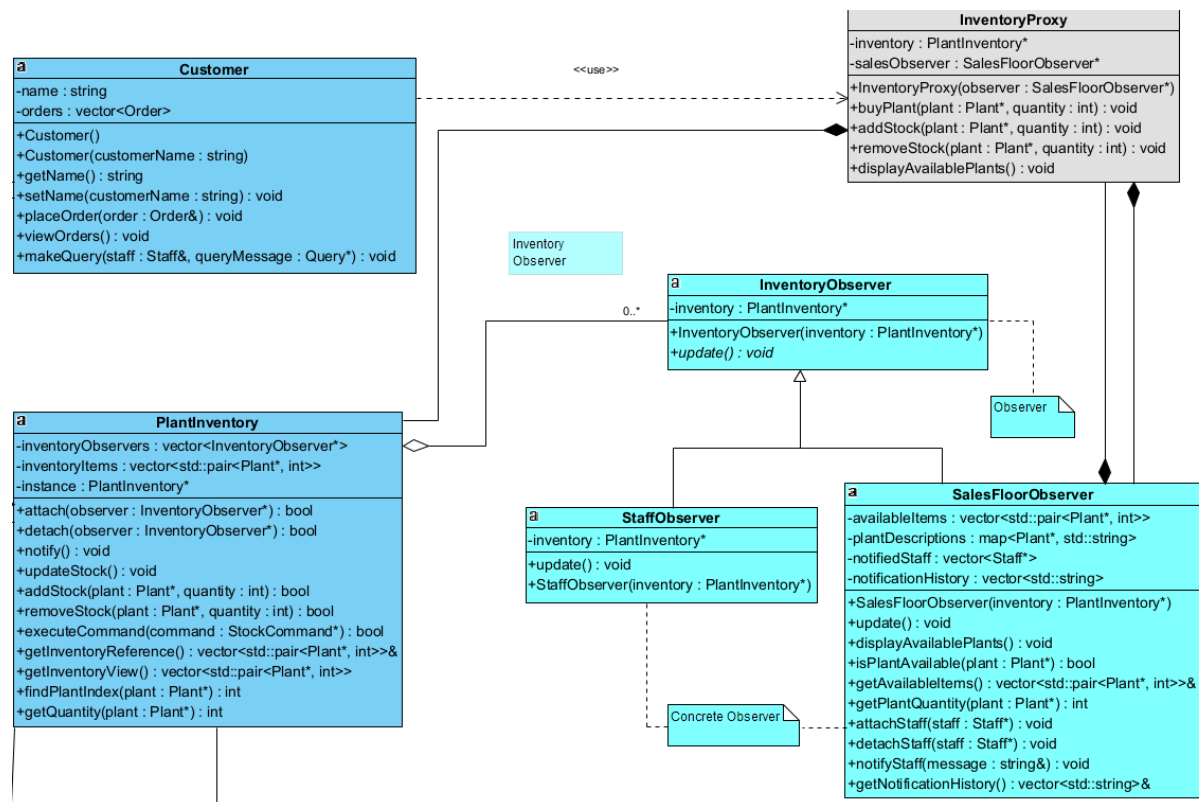
This pattern is chosen for several reasons:

- Decoupling Sender and Receiver: The pattern allows the customer to send a request without knowing which specific employee will handle it, promoting flexibility and reducing tight coupling between components.
- The system allows staff to assist customers with queries. Each staff member has different roles and what it is able to handle.

Justification:

The Chain of Responsibility pattern was chosen to enable flexible and dynamic handling of customer requests. By decoupling the sender (customer) from the receiver (staff), the system can automatically determine which staff member is best suited to handle a request without requiring direct dependencies. This promotes scalability and maintainability, as employees can be added, removed, or reassigned without disrupting the request-handling process.

9 Proxy (Customer Interaction with Staff)



The Proxy pattern is utilized to add an extra layer of control and security to between the customer interactions with access certain actions only staff can do such as only staff can add and remove to `PlantInventory`.

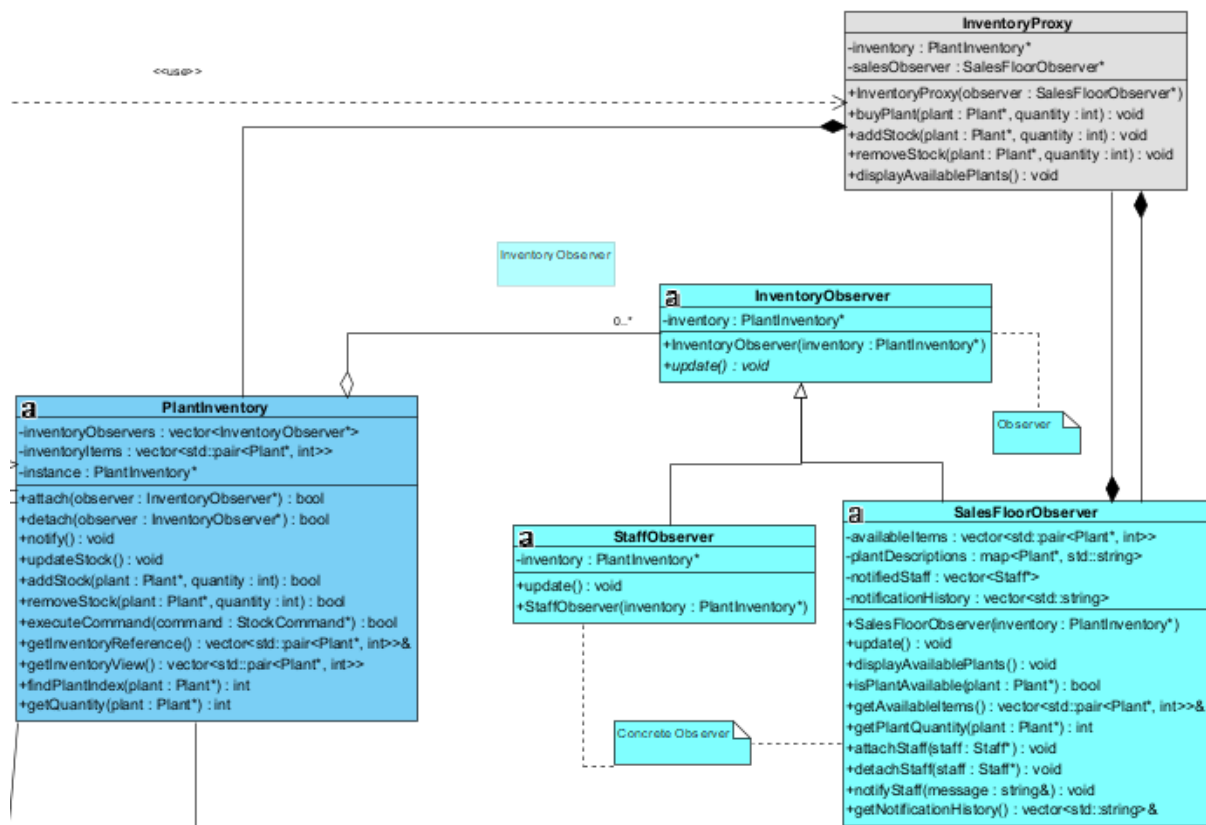
This patterns is chosen for several reasons:

- **Access Control:** The Protection Proxy restricts specific actions such as adding or removing items from the `PlantInventory`, to authorized staff members only, ensuring customers cannot perform these operations.
- **Enhanced Security Layer:** By acting as an intermediary between customers and the system, the proxy adds an extra layer of security that enforces permissions and prevents unauthorized access to sensitive functionality.

Justification

The Proxy pattern was chosen to introduce an additional layer of control and security between customers and the system. By acting as an intermediary, the proxy ensures that only authorized staff members can perform sensitive operations such as adding or removing items from the `PlantInventory` essentially customers are restricted from direct access.

10 Observer design patterns for Plant Inventory



The Observer design pattern is used to notify staff and other components of changes to the plant inventory.

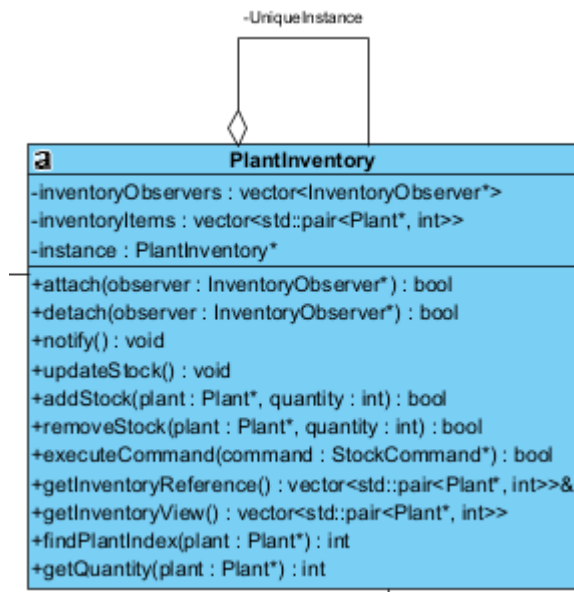
This pattern is chosen for several reasons:

- **Decoupling:** The Observer pattern decouples the **PlantInventory** class from the **StaffObserver** and **SalesFloorObserver** classes, allowing for greater flexibility and extensibility. This decoupling enables the **PlantInventory** class to focus solely on managing the inventory, without worrying about the specific details of how each observer will react to changes.
- **Real-time Updates:** The Observer pattern enables real-time updates to staff and other components when the inventory changes, ensuring that everyone has the latest information. This ensures that staff can provide accurate information to customers.

Justification

The Observer pattern is used to notify staff and other components of changes to the plant inventory, allowing for real-time updates and decoupling the inventory management from the notification process. This enables greater flexibility, extensibility, and accuracy in inventory information, ultimately improving customer service.

11 Singleton design patterns for Plant Inventory



The Singleton design pattern is used to ensure that there is only one instance of the **PlantInventory** class.

This pattern is chosen for several reasons:

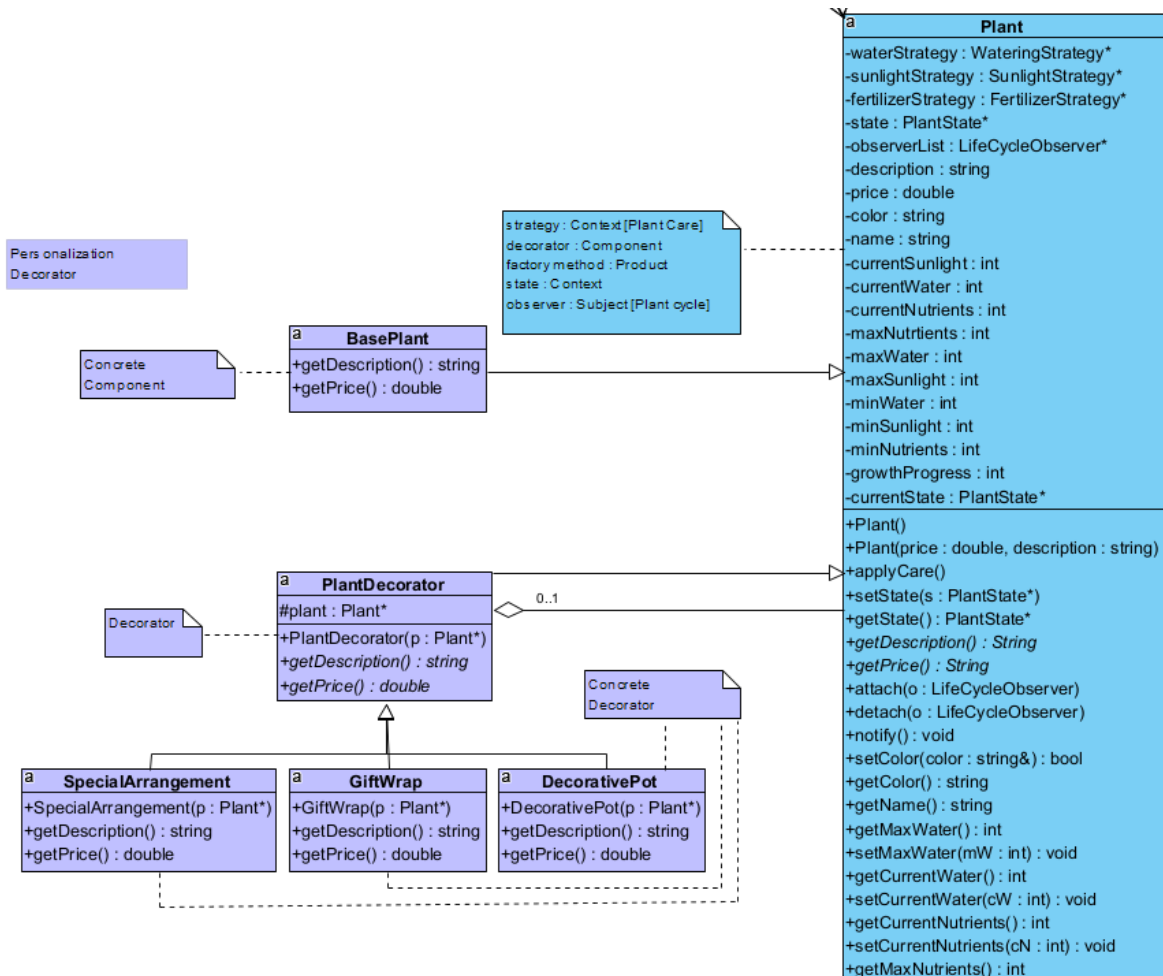
- **Global Access:** The Singleton pattern provides global access to the **PlantInventory** instance, making it easy for staff and other components to access the inventory.
- **Resource Efficiency:** By ensuring that there is only one instance of the **PlantInventory** class, the Singleton pattern helps to conserve resources and prevent unnecessary duplication of effort.
- **Simplified Management:** The Singleton pattern simplifies the management of the **PlantInventory** instance, as it provides a single point of access and control. This makes it easier to manage the inventory and ensure that it is accurate and up-to-date.

Justification

The Singleton pattern is used to ensure a single, accessible instance of the **PlantInventory** class, providing resource efficiency, simplified management, and easy access to the inventory for staff and other components, ultimately supporting accurate and up-to-date inventory management.

CUSTOMER AND SALES

12 Plant add-ons with Decorator design pattern



The Decorator pattern is used to add various extras to a plant purchase, such as decorative pots, gift wrapping, and special arrangements.

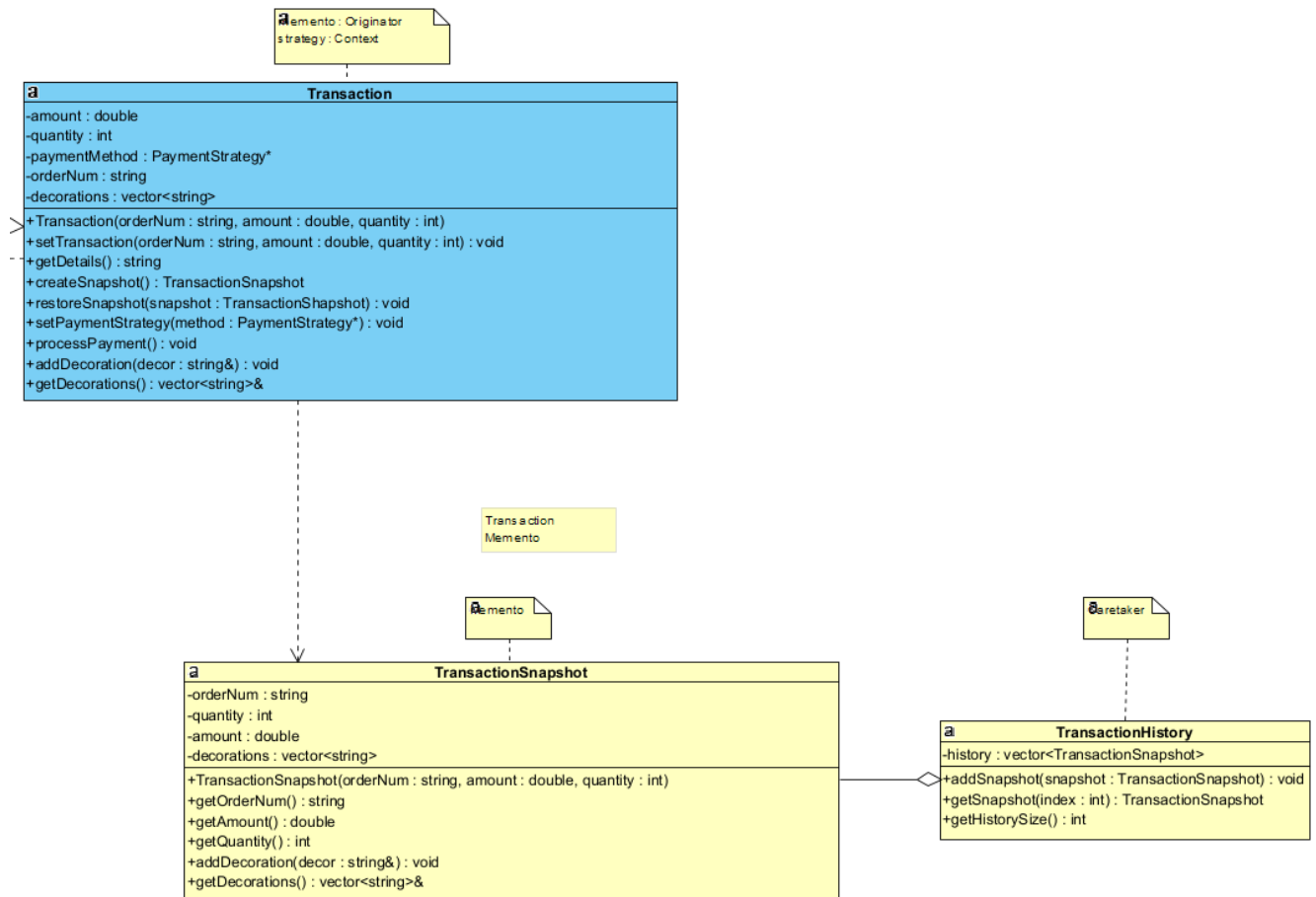
This pattern is chosen for several reasons:

- **Flexibility:** The Decorator pattern allows us to add or remove extras dynamically, without modifying the underlying plant class.
- **Reusability:** The Decorator pattern promotes code reuse by allowing us to create multiple decorators that can be used with different plant classes.
- **Adding extras is optional:** The Decorator pattern allows us to add extras only when needed, without affecting the underlying plant class.
- **Multiple extras can be added:** The Decorator pattern allows us to add multiple extras to a plant purchase, such as a decorative pot and gift wrapping.
- **Easy to extend:** The Decorator pattern makes it easy to add new extras in the future, without modifying the existing code.

Justification

The Decorator pattern is perfectly suited for adding extras to plant purchases, as it provides a flexible and scalable solution that preserves the integrity of the underlying plant class. By allowing for dynamic addition or removal of extras like decorative pots, gift wrapping, and special arrangements, this pattern enables a wide range of customization options without code duplication or modification to existing classes. Its modular design promotes code reuse, making it easy to introduce new extras or combine multiple extras, thereby enhancing the overall customer experience.

13 Transaction history with Memento design pattern



The Memento pattern is used here to store the transaction history. Each transaction snapshot is a memento that captures the state of the transaction at a particular point in time. This allows us to store the transaction history and restore previous transactions if needed.

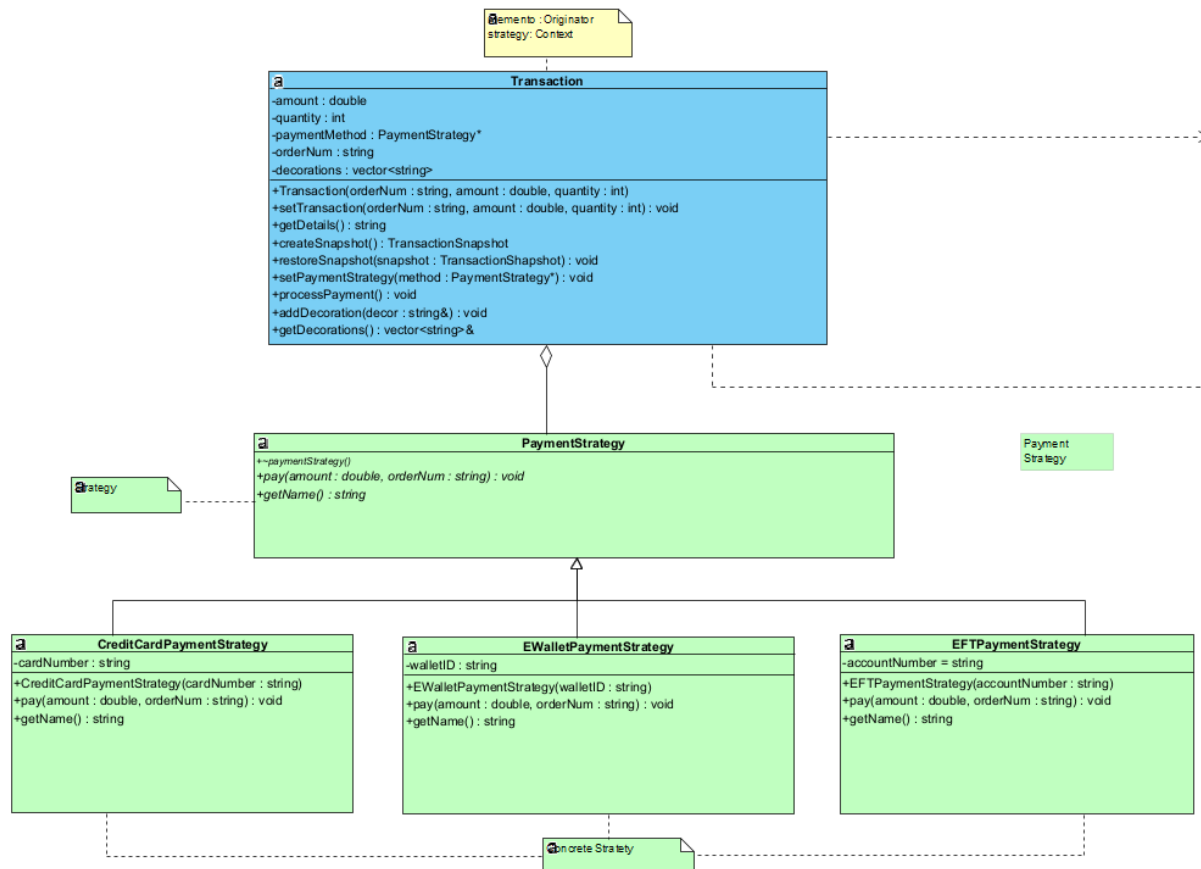
This pattern is chosen for several reasons:

- **Preserving transaction history:** The Memento pattern allows us to store the transaction history and retrieve previous transactions.
- **Flexibility:** The Memento pattern makes it easy to add or remove transactions from the history.
- **Encapsulation:** The transaction history is encapsulated within the memento, making it easier to manage and maintain.
- **Transaction logging:** All transactions are logged and stored in a memento, allowing for easy retrieval and analysis.
- **Effortless repeat purchases:** Customers can quickly reorder previous transactions using the Memento pattern, enhancing convenience and user satisfaction.

Justification

The Memento pattern is beneficial for the nursery as it provides a reliable and efficient way to store and manage transaction history. By preserving transaction history and allowing for easy retrieval and restoration of previous transactions, the nursery can maintain accurate records, streamline repeat purchases, and enhance customer convenience. This leads to improved customer satisfaction, operational efficiency supporting the nursery's business goals.

14 Payment methods with Strategy design pattern



The Strategy pattern is used here to implement different payment strategies (e.g., credit card, EFT). This allows us to add or remove payment strategies without modifying the Transaction class.

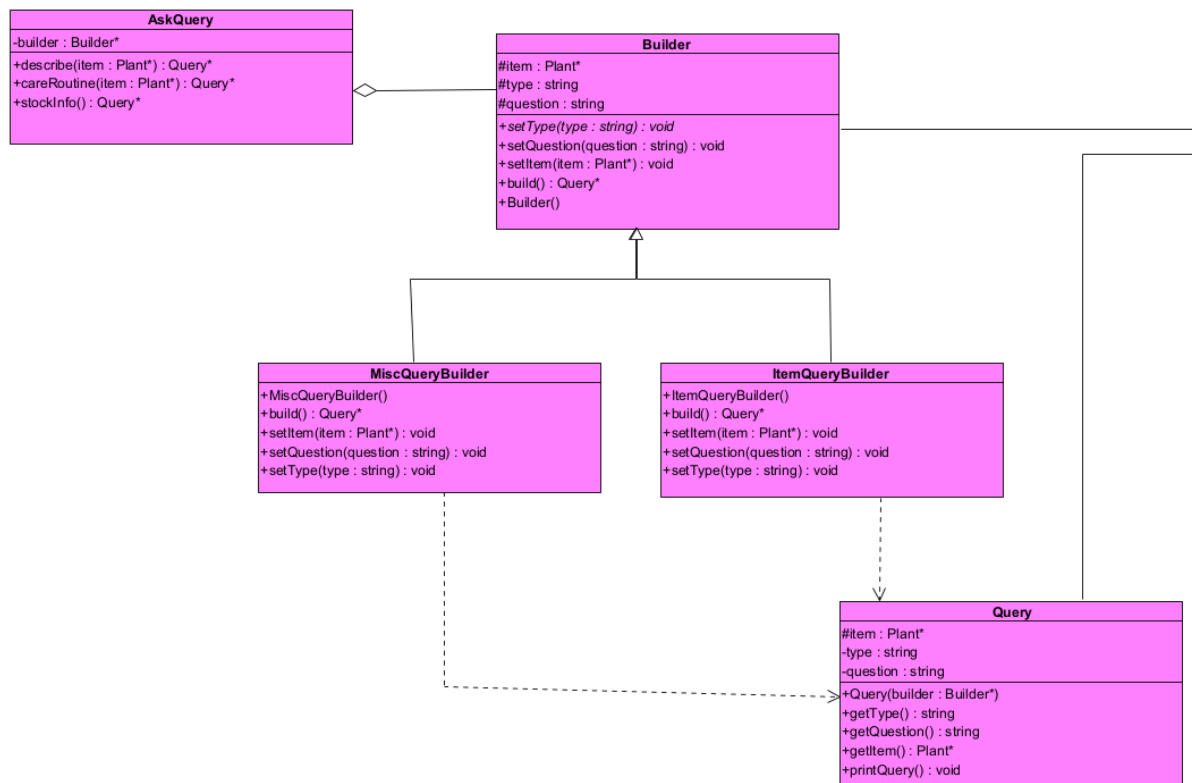
This pattern is chosen for several reasons:

- **Flexibility:** The Strategy pattern makes it easy to add or remove payment strategies.
- **Decoupling:** The Strategy pattern decouples the payment strategy from the Transaction class.
- **Dynamic Strategy Selection:** The Strategy pattern allows us to choose a payment strategy at runtime, which is particularly useful in this scenario where customers can select their preferred payment method. This dynamic selection enables us to handle different payment strategies without having to know the specific strategy at compile time.

Justification

The Strategy pattern is well-suited for the nursery's payment system, as it provides flexibility and scalability in managing different payment strategies, such as credit card and EFT. By decoupling the payment strategy from the Transaction class, the nursery can easily add or remove payment methods as needed, accommodate changing customer preferences, and ensure seamless transactions. This dynamic approach enables the nursery to stay adaptable and responsive to customer needs.

15 Queries with Builder Pattern



The Builder pattern is used to construct complex queries enabling customers to ask queries which staff will handle and answer.

This pattern is chosen for several reasons:

- **Readability and Flexibility:** The Builder pattern provides a fluent, readable interface for constructing complex objects. It avoids the need for constructors with a large number of parameters.
- **Handles Optional Parameters:** Many aspects of a plant query are optional. The Builder pattern gracefully handles this, allowing clients to specify only the criteria they care about.
- **Immutability:** The built query object can be made immutable, which is safe to share and execute.
- **Separation of Concerns:** The Builder pattern separates the construction of queries from their representation, making the code more modular and easier to maintain.
- **Flexibility:** The Builder pattern allows us to create different types of queries (e.g., item queries, misc queries) without modifying the underlying code.
- **Reusability:** The Builder pattern promotes code reusability by allowing us to use the same builder class to create different queries.

Justification:

The Builder pattern is justified for use in the nursery system because it provides a readable and flexible interface for constructing complex queries, handles optional parameters, separates concerns, and allows for flexibility and reusability. By using this pattern, the nursery system can improve the overall shopping experience for customers.

GITHUB BRANCH STRATEGY

Our team follows a **feature branch-based** Git workflow built around a central development branch. Each new feature or bug fix is developed in its own branch created from development, ensuring isolated, focused work. For grouped testing and integration, we create merge branches as children of development, where multiple related features are combined and validated before merging back. Once testing is complete, the merge branch is integrated into development, and after overall stabilisation, the code is promoted to main for production. This structure helps maintain code stability, supports parallel development, and streamlines testing and release management.

main branch

- The main branch contains production-ready code.
- Only thoroughly tested and approved changes from the development branch are merged here.
- It represents the live version of the application and is used for releases and deployments.

component branches

- Component branches are created from the development branch for individual features, enhancements, or bug fixes.
- Each branch is named according to its purpose, such as after its design pattern and what category (Greenhouse, Staff, Customer & Sales) it belongs to or what is being merged or tested or being worked on (UML).
- Developers work independently on these branches to avoid interfering with the main codebase.

workflow

- Developers create component branches from the development branch to work on specific tasks.
- Related features are sometimes grouped and merged into a temporary merge branch for testing and validation.
- Once all tests pass, the merge branch is merged back into the development branch.
- After all required features for a release are stable, the development branch is merged into the main branch.

merging into main

- The development branch is periodically merged into the main branch after integration and QA testing.
- The merge is typically done through a pull request to ensure review and approval.
- Only stable, production-ready code should reach the main branch.

branch protection

- The main branch is protected to prevent direct commits.
- All changes must go through pull requests and be approved before merging.
- Continuous integration checks and required reviews are enforced to maintain code quality and stability.

name convention

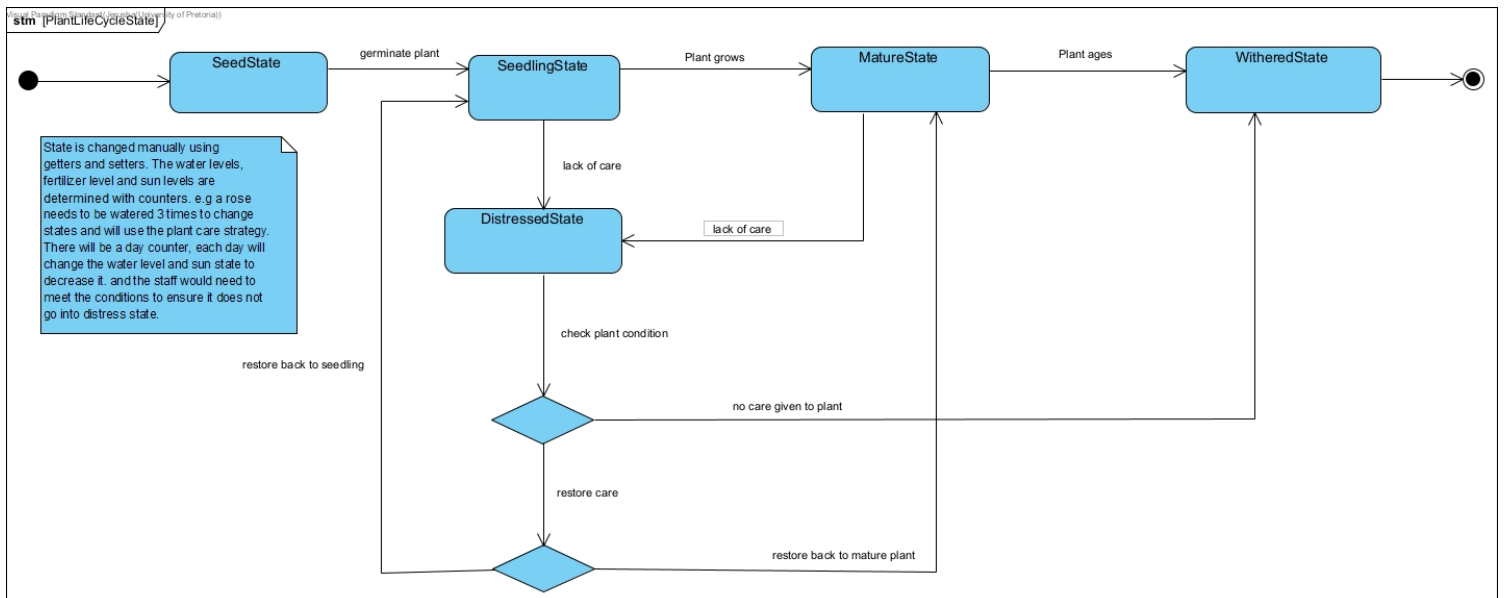
- The project was divided into three main components: Greenhouse, Staff, Customer and Sales (C&S).
- Branches were named using the component and the design pattern applied, for example, Greenhouse-Strategy.
- Merge branches combining multiple components followed a format such as Greenhouse-C&S-merge.
- Feature or testing branches were named based on their specific purpose for example, DemoTesting.

GITHUB ACTIONS

In our project, we implemented GitHub Actions to automate the linting, testing, and build processes as part of our CI/CD pipeline.

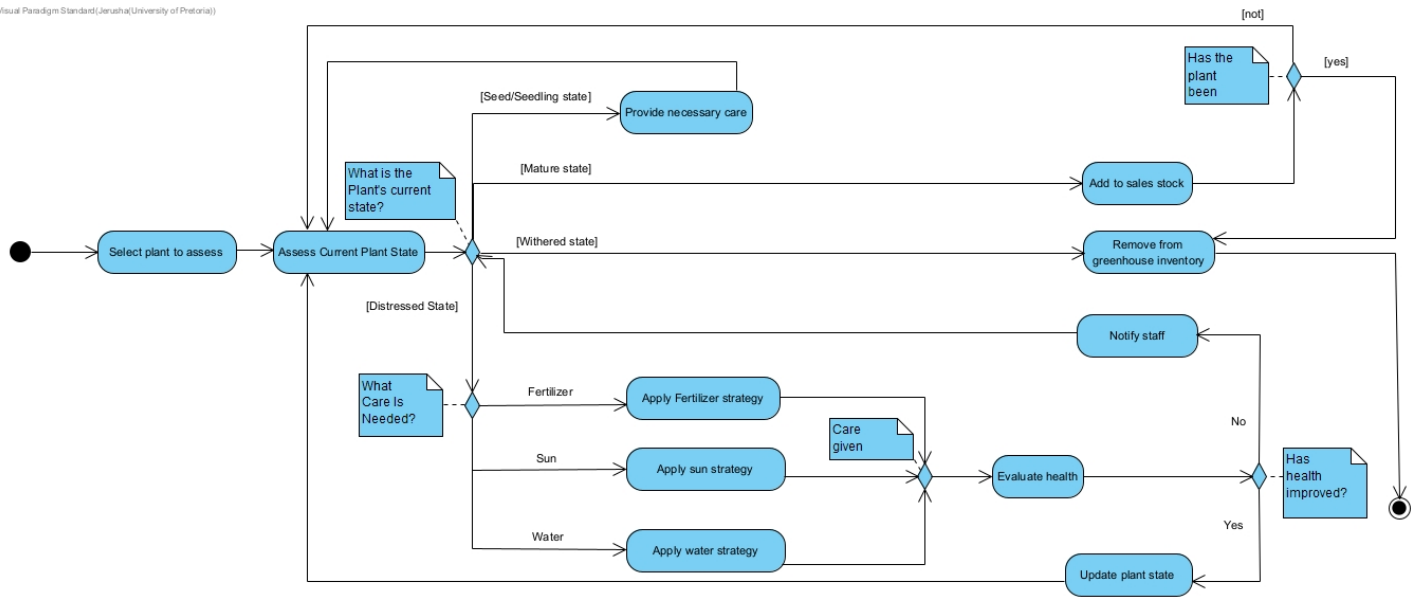
- A GitHub Actions workflow file was created in the .github/workflows directory.
- The pipeline is triggered automatically whenever a pull request is created or code is pushed to the development or main branch.
- The workflow includes a linting step to check code style and formatting issues, ensuring consistency across the codebase.
- It also runs automated unit tests to validate functionality and prevent regressions before merging.
- After successful linting and testing, the pipeline proceeds to build the application to verify that the code compiles and deploys correctly.
- This setup helps maintain code quality, catch errors early, and streamline the integration process without manual intervention.

STATE DIAGRAM



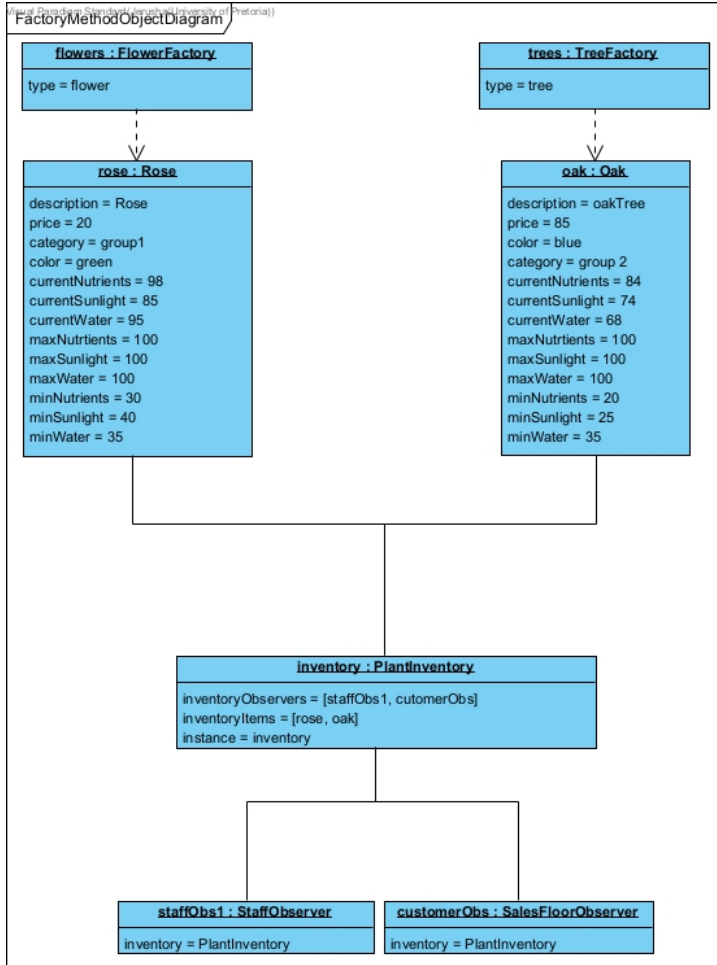
ACTIVITY DIAGRAM

Visual Paradigm Standard (Jerusalem/University of Pretoria))

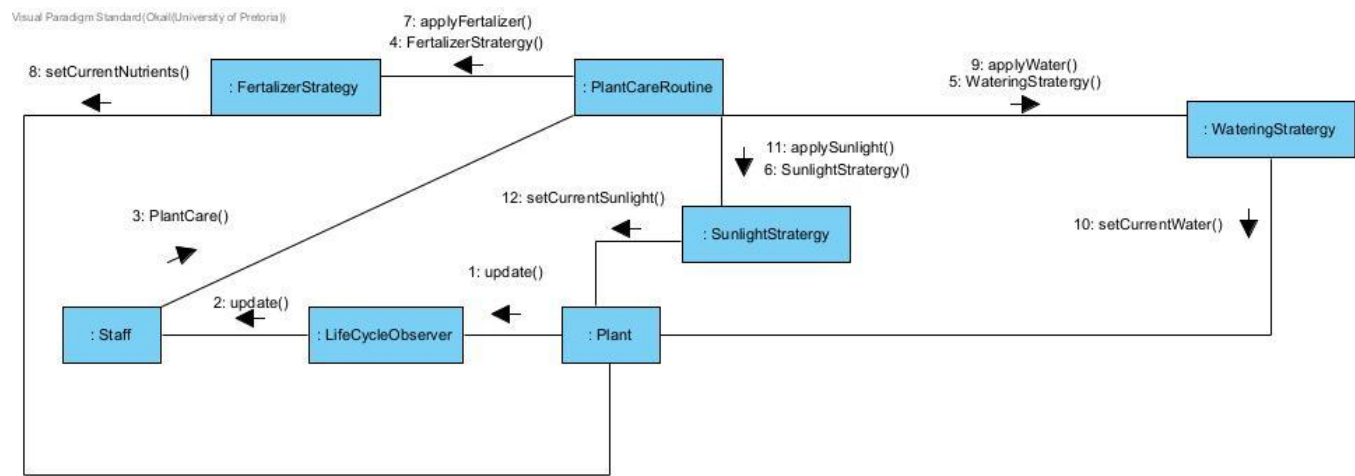


OBJECT DIAGRAM

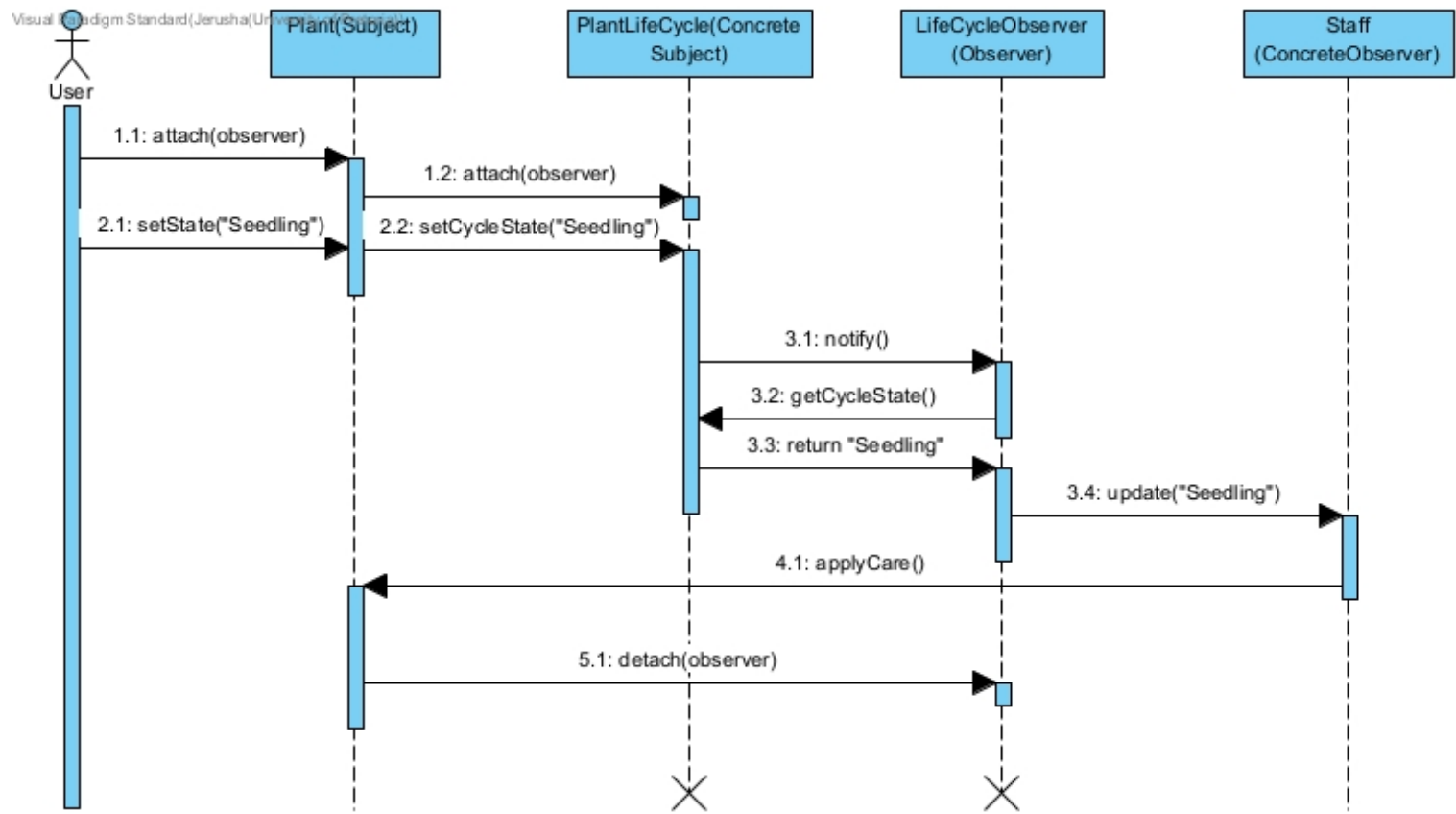
Visual Paradigm Standard (Jerusalem/University of Pretoria))



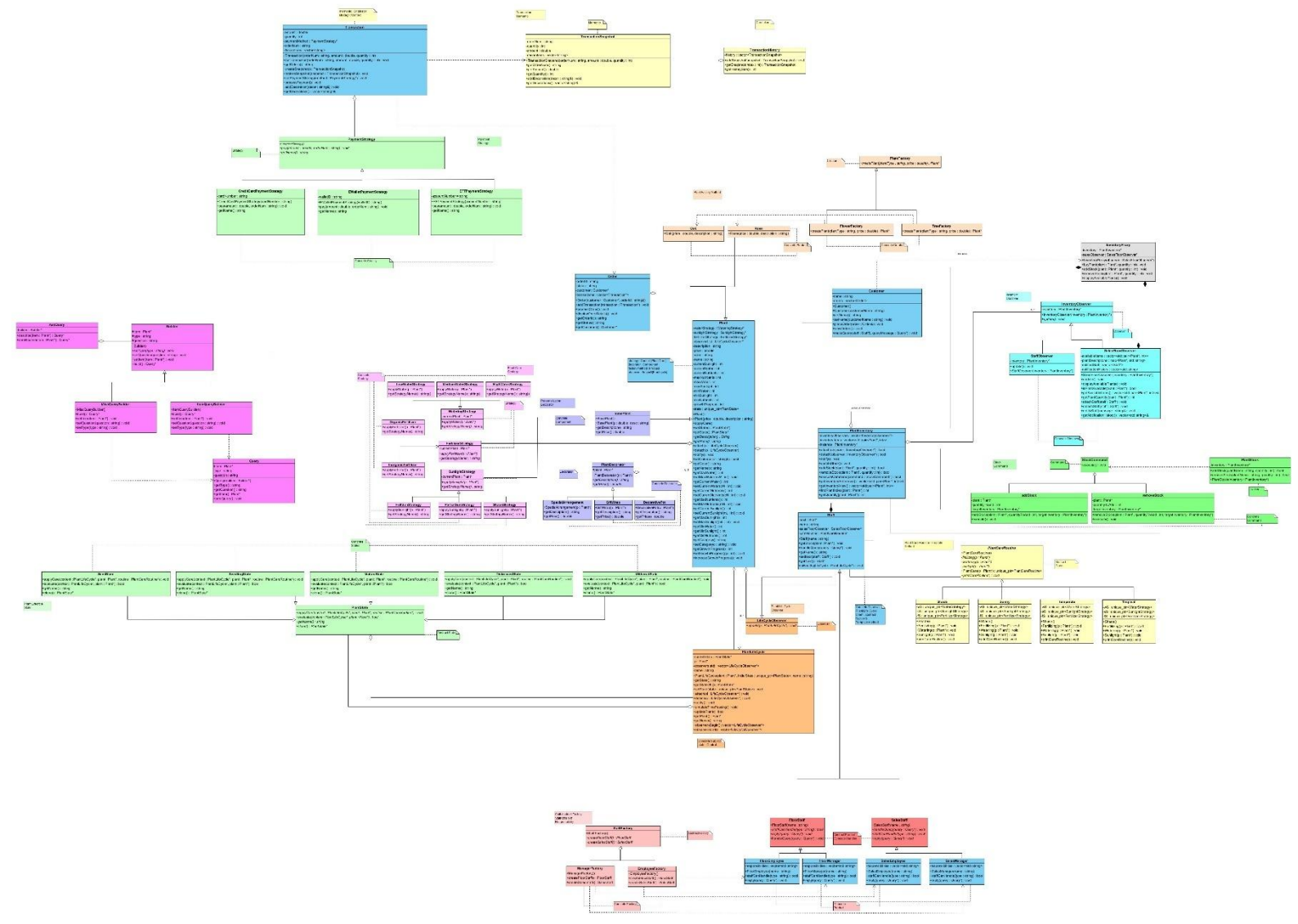
COMMUNICATION DIAGRAM



SEQUENCE DIAGRAM



SYSTEM CLASS DIAGRAM



HIGH LEVEL ACTIVITY DIAGRAM

